A01 · Registro de participantes — Ampliación

DWEC · 2.º DAW · UD2 DOM y eventos
Trabajo individual

Puedes hacer uso de los enlaces a internet de los apuntes, pero no realizas búsquedas ni uses la IA.

1. Reto 1 Evitar correos duplicados

El correo electrónico identifica a cada participante. Impide registrar dos veces la misma dirección, aunque cambien las mayúsculas o aparezcan espacios al principio o al final.

Requisitos

- Mantén las validaciones de la práctica original: campos obligatorios, formato de correo y nombre de al menos dos caracteres después de aplicar trim().
- Antes de añadir una fila, compara el correo con todos los participantes registrados, incluidos los que queden ocultos al buscar.
- Para esta actividad, compara las direcciones aplicando trim().toLowerCase() a ambos valores.
- Si ya existe, muestra «Este correo ya está registrado.». No añadas la fila, no cambies el contador y conserva los campos para corregirlos.
- Si no existe y los datos son válidos, realiza el alta con el comportamiento original.

Pista

Puedes guardar el correo normalizado en li.dataset.email al crear cada participante.

2. Reto 2 Eliminar individualmente

Añade un botón «Eliminar» a cada participante. Al pulsarlo, debe desaparecer únicamente su fila y actualizarse el contador.

```
Participantes (2)

Ana — ana@example.com      [Eliminar]
Luis — luis@example.com    [Eliminar]
```

- Crea el botón mediante document.createElement() y establece type="button" para que no envíe el formulario.
- Separa el texto del participante y sus botones en nodos distintos. Usa textContent para el nombre y el correo.
- Asocia el evento click a la eliminación de la fila correspondiente. Muestra «Participante eliminado.».
- Actualiza el total después de eliminar. Si borras el último participante, debe quedar en 0.
- El botón «Vaciar lista» debe seguir funcionando. Un correo eliminado debe poder registrarse de nuevo.

Pista

Puedes añadir un listener al crear cada botón y conservar la referencia a su <li>. El método remove() elimina ese elemento. También puedes delegar el evento en la lista si sabes identificar correctamente el botón pulsado y su fila.

3. Reto 3 Buscar participantes

Incorpora un buscador que filtre la lista por nombre o correo mientras escribes. Añade este HTML fuera del formulario de registro, antes de la lista:

```
<label for="buscar">Buscar participantes</label>
<input
  type="search"
  id="buscar"
  placeholder="Escribe un nombre o correo"
>
<p id="resultadoBusqueda" aria-live="polite"></p>
```

Requisitos

- Escucha el evento input. El filtrado debe funcionar al escribir, borrar o pegar texto, sin pulsar un botón.
- Busca coincidencias parciales en el nombre o en el correo, sin distinguir mayúsculas y minúsculas. Elimina los espacios de los extremos de la búsqueda.
- Oculta las filas que no coincidan y muestra las que sí coincidan. No elimines participantes para filtrar.
- Si la búsqueda está vacía, muestra todos los participantes.
- Si hay participantes pero ninguno coincide, muestra «No hay participantes que coincidan con la búsqueda.». Si la lista está vacía, muestra «No hay participantes registrados.».
- El contador total seguirá mostrando cuántos participantes hay registrados, aunque algunos estén ocultos.
- Reaplica el filtro después de añadir, eliminar, vaciar o editar. Los botones deben actuar sobre el participante correcto con una búsqueda activa.

Pistas

Guarda por separado el nombre y el correo, usa para ello dataset. Evita buscar sobre el texto de los botones.

```
fila.hidden = !coincide;
```

La propiedad hidden permite ocultar la fila sin borrarla. Centraliza el filtrado en una función que puedas ejecutar tanto desde input como después de modificar la lista.

Comprobación rápida

Registra a Ana y a Luis. Escribe «ANA»: solo debe verse Ana. Busca «example.com»: deben verse ambos. Busca «zzz»: no debe verse ninguna fila y debe aparecer el aviso. Borra la búsqueda: deben volver a verse ambos y el total debe seguir siendo 2.

4. Reto 4 Editar un participante

Añade un botón «Editar» a cada fila. Reutiliza el formulario para modificar los datos de un participante existente.

- Al pulsar «Editar», carga el nombre y el correo en sus campos. Cambia el texto de «Registrar» por «Guardar cambios» e indica qué participante estás editando.
- Conserva una referencia a la fila que se está editando. Al guardar, modifica esa fila en lugar de crear otra; el total no debe variar.
- Aplica las mismas validaciones del alta. Al comprobar duplicados, excluye la propia fila: conservar el correo actual debe ser válido.
- Si hay un error, conserva el modo de edición y los datos introducidos. No modifiques la fila hasta que toda la validación sea correcta.
- Añade un botón «Cancelar edición» con type="button". Debe volver al modo de alta sin modificar al participante.
- Después de guardar, actualiza el texto y los datos utilizados por el buscador, reaplica el filtro y muestra «Participante actualizado.». Limpia el formulario y recupera el botón «Registrar».
- Si eliminas la fila que estás editando o vacías la lista, cancela también la edición para no conservar una referencia inválida.

Pista

Una variable inicializada a null puede guardar la referencia al participante en edición. Si contiene una fila, submit actualiza; si vale null, registra. Centraliza en una función la salida del modo de edición.

5. Reto 5 Confirmación y experiencia de usuario

- Pide confirmación antes de eliminar un participante y antes de vaciar una lista con datos. Puedes usar window.confirm().
- Si se cancela, conserva la lista, el contador, el filtro y cualquier edición en curso.
- Si la lista ya está vacía, el botón «Vaciar lista» debe funcionar sin error y sin pedir una confirmación innecesaria.
- Utiliza mensajes claros para altas, errores, modificaciones y eliminaciones. Retira mensajes antiguos cuando dejen de tener sentido.
- Cuida el foco: al comenzar una edición, lleva al campo nombre; tras registrar o guardar, devuelve también el foco a ese campo. Al borrar una fila, sitúalo en un control que siga existiendo.
- Permite usar la aplicación con teclado. Mantén botones y etiquetas comprensibles; puedes añadir aria-live="polite" al párrafo mensaje.

6. Requisitos comunes y herramientas

- Utiliza const y let, addEventListener() y funciones con nombres claros. Evita repetir bloques de actualización del contador o del filtro.
- Usa createElement(), textContent y append() para construir las filas. No insertes datos introducidos por el usuario como HTML.
- Mantén event.preventDefault() en submit. Un envío válido debe producir exactamente un alta o una actualización.
- Conserva reportValidity(), el saludo mediante input, la limpieza de campos tras un alta válida y los comentarios breves sobre las partes principales.
- Comprueba el funcionamiento en el navegador y revisa la consola después de cada reto. Utiliza únicamente datos ficticios.

Herramientas útiles: dataset para guardar datos en una fila; remove() para eliminarla; hidden para ocultarla; children.length para contar todos los participantes, incluidos los ocultos; focus() para mover el foco; reset() para reiniciar el formulario y replaceChildren() para vaciar la lista.

7. Tabla de pruebas

Realiza las pruebas obligatorias y las avanzadas de los retos que implementes. Registra el resultado obtenido en pruebas.txt; si algo falla, explica el fallo.

Nivel	Prueba	Resultado esperado
Base	Repetir las pruebas de la práctica inicial.	Siguen funcionando saludo, validación, alta con Enter, contador y vaciado.
R1	Registrar Ana y repetir su correo con mayúsculas y espacios exteriores.	Rechaza el duplicado; conserva los campos y el total.
R1	Intentar duplicar un participante oculto por el filtro.	También rechaza ese duplicado.
R2	Eliminar una fila intermedia y luego la última.	Borra solo la seleccionada; actualiza el total hasta 0.
R2	Volver a registrar un correo eliminado.	Permite el alta.
R3	Buscar por nombre y por fragmento de correo; borrar el filtro.	Muestra las coincidencias y después toda la lista; no cambia el total.
R3	Buscar sin coincidencias; añadir o borrar con filtro activo.	Aviso correcto y filtro actualizado; no pierde registros.
R4	Guardar sin cambiar el correo; después usar el de otra fila.	Permite el primer caso y rechaza el duplicado sin alterar la lista.
R4	Cancelar la edición; eliminar la fila editada o vaciar.	No guarda cambios al cancelar; sale del modo edición si desaparece la fila.
R5	Cancelar y aceptar las confirmaciones; recorrer con Tab.	Cancelar conserva el estado; aceptar ejecuta la acción; foco utilizable.

9. Entrega

Entrega la carpeta de tu aplicación ampliada comprimida en un archivo ZIP. Consulta la sección general de Moodle para nombrar la actividad.

```
registro-participantes/
  index.html
  app.js
  pruebas.txt
```

- index.html: estructura completa de la aplicación con el buscador y las mejoras realizadas.
- app.js: código de la aplicación inicial y de la ampliación, con comentarios breves en las partes principales.
- pruebas.txt: retos completados, pruebas realizadas y resultados. Indica expresamente si has realizado los retos 4 y 5.

Para cada prueba utiliza este formato:

```
Prueba:
Datos o acción:
Resultado esperado:
Resultado obtenido:
```

Antes de comprimir

- Abre index.html desde la carpeta que entregarás y comprueba que carga app.js.
- Verifica que los retos 1–3 funcionan juntos, no solo por separado.
- Comprueba que no aparecen errores en la consola y que el ZIP contiene los tres archivos.
- Si implementas una ampliación avanzada, incluye también sus pruebas.